

Tuyển Tập Bài Tập: Bảng Thưa & Xử Lý Truy Vấn Tĩnh Đoạn $O(1)$ (Sparse Table & RMQ)

(Curated Competitive Programming Problems on Sparse Table, RMQ & Binary Lifting)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 **Duc-Minh Vu**. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Bảng Tổng Hợp 16 Bài Tập Tuyển Chọn

STT	Mã Bài	Tên Bài Toán	Nguồn	Độ Khó	Chủ Đề Trọng Tâm
1	CSES 1647	Static Range Minimum Queries	CSES	☆☆☆	RMQ chuẩn mực, xây dựng $O(N \log N)$, truy vấn $O(1)$
2	SPOJ RMQSQ	Range Minimum Query	SPOJ	☆☆☆	Khởi động cấu trúc bảng thưa 0-indexed
3	CF 1548B	Integers Have Friends	Codeforces	☆☆☆	Sparse Table trên mảng hiệu $\gcd(A_i - A_{i-1}) > 1$
4	CSES 1687	Company Queries I	CSES	☆☆☆	Binary Lifting tìm tổ tiên thứ K trên cây
5	CF 475D	CGCDSSQ	Codeforces	☆☆☆☆	Sparse Table GCD + Tìm kiếm nhị phân đếm đoạn con
6	CF 689D	Friends and Subsequences	Codeforces	☆☆☆☆	Hai Sparse Table Min & Max song song, nhị phân hai biên
7	CF 514D	R2D2 and Droid Army	Codeforces	☆☆☆☆	5 Sparse Table Max kết hợp kỹ thuật Hai con trỏ
8	SPOJ THRBL	Catapult that ball	SPOJ	☆☆☆☆	Kiểm tra giá trị cực đại trên đoạn $[A + 1, B - 1] \leq H[A]$
9	CSES 1688	Company Queries II	CSES	☆☆☆☆	Tìm LCA $O(1)$ qua Euler Tour RMQ
10	ABC 254F	Rectangle GCD	AtCoder	☆☆☆☆	Ma trận $A_i + B_j$, tách gcd thành 2 Sparse Table 1D

STT	Mã Bài	Tên Bài Toán	Nguồn	Độ Khó	Chủ Đề Trọng Tâm
11	CF 1454F	Array Partition	Codeforces	★★★★★	Phân hoạch 3 đoạn $\max_1 = \min_2 = \max_3$ bằng Sparse Table
12	CF 1709E	XOR Tree	Codeforces	★★★★★	Biến đổi khoảng cách trên cây, LCA $O(1)$ và DSU on Tree
13	SPOJ RPAR	Reduce the Array	SPOJ	★★★★★	Disjoint Sparse Table (DST) tính tổng / tích đa thức $O(1)$
14	CF 1304E	1-Trees and Queries	Codeforces	★★★★★	Khoảng cách cây qua LCA $O(1)$, kiểm tra chu trình chẵn/lẻ
15	CF 1779F	Even Subarrays	Codeforces	★★★★★	Sparse Table AND/OR kết hợp đếm số lượng ước lẻ
16	CF 1117G	Recursive Queries	Codeforces	★★★★★	Sparse Table Index dựng Cartesian Tree $O(N) \rightarrow$ Offline BIT

💡 Phân Tích & Hướng Dẫn Chi Tiết (3-Tier Hints)

Bài 1: CSES 1647 — Static Range Minimum Queries

- **Đề bài:** Cho mảng A gồm N số nguyên ($N, Q \leq 2 \times 10^5$). Trả lời Q truy vấn tìm giá trị nhỏ nhất trên đoạn $[L, R]$.
- **Gợi ý 1 (Nhận xét bản chất):**
 - Mảng dữ liệu tĩnh hoàn toàn, không có thao tác thêm/sửa/xóa.
 - Phép tìm giá trị nhỏ nhất $\min(x, y)$ có tính chất lũy đẳng ($\min(x, x) = x$).
- **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Xây dựng bảng thưa `st[k][i]` với $k \in [0, 18]$ và $i \in [0, N - 1]$.
 - Thời gian tiền xử lý: $O(N \log N)$.
 - Với mỗi truy vấn $[L, R]$, tính $k = \lfloor \log_2(R - L + 1) \rfloor$ trong $O(1)$ bằng `std::__lg` hoặc `std::bit_width`.
 - Đáp án: $\min(st[k][L], st[k][R - 2^k + 1])$.
- **Gợi ý 3 (Độ phức tạp & Cạm bẫy):**
 - Thời gian: $O(N \log N + Q)$, Bộ nhớ: $O(N \log N)$.
 - Cạm bẫy: Chỉ số 1-indexed của đề bài cần đổi về 0-indexed hoặc khởi tạo mảng kích thước $N + 1$.

Bài 2: SPOJ RMQSQ — Range Minimum Query

- **Đề bài:** Cho mảng N phần tử ($N \leq 10^5$) và $Q \leq 10^5$ truy vấn $0 \leq i \leq j < N$. Tìm $\min(A[i \dots j])$.
- **Gợi ý 1 (Nhận xét bản chất):**
 - Bài toán cơ bản nhất để kiểm tra cấu trúc Sparse Table thuần túy.

- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Cài đặt cấu trúc dữ liệu `SparseTable` với thứ tự `st[k][i]` để đảm bảo tính liên tục bộ nhớ (cache-friendly).
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian: Tiền xử lý $O(N \log N)$, truy vấn $O(1)$.
- Cận bậc: Cần thận giới hạn $i = j$ (đoạn có độ dài 1).

Bài 3: Codeforces 1548B — Integers Have Friends

- **Đề bài:** Cho mảng A gồm N phần tử ($N \leq 2 \times 10^5$, $A_i \leq 10^{18}$). Tìm độ dài đoạn con liên tiếp dài nhất $A[L \dots R]$ sao cho tồn tại số nguyên $m \geq 2$ thỏa mãn:

$$A_L \equiv A_{L+1} \equiv \dots \equiv A_R \pmod{m}$$

- **Gợi ý 1 (Nhận xét bản chất):**
- Điều kiện đồng dư tương đương với: $m \mid |A_i - A_{i-1}|$ với mọi $i \in [L + 1, R]$.
- Tồn tại $m \geq 2$ khi và chỉ khi:

$$\gcd(|A_{L+1} - A_L|, |A_{L+2} - A_{L+1}|, \dots, |A_R - A_{R-1}|) \geq 2$$

- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Tạo mảng hiệu D gồm $N - 1$ phần tử: $D_i = |A_{i+1} - A_i|$.
- Dùng Sparse Table tính gcd trên mảng D . Phép gcd là phép toán lũy đẳng nên truy vấn trong $O(1)$.
- Dùng kỹ thuật Hai con trỏ (Two Pointers) hoặc Tìm kiếm nhị phân trên cận phải: Với mỗi L , tìm R xa nhất sao cho $\gcd(D[L \dots R]) > 1$.
- Độ dài đoạn kết quả là $(R - L + 1) + 1$.
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian: $O(N \log N \log(\max A))$, Bộ nhớ: $O(N \log N)$.
- Cận bậc: $D_i = 0$ khi hai phần tử cạnh nhau bằng nhau, $\gcd(0, x) = x$. Dùng kiểu `long long`.

Bài 4: CSES 1687 — Company Queries I

- **Đề bài:** Cho cây có gốc với N nhân viên ($N, Q \leq 2 \times 10^5$). Trả lời Q truy vấn: Tìm cấp trên thứ K của nhân viên u (hoặc -1 nếu không tồn tại).
- **Gợi ý 1 (Nhận xét bản chất):**
- Đây là kỹ thuật Nhảy nhị phân (Binary Lifting) trên cây, có cùng nguyên lý phân rã lũy thừa 2 như Sparse Table.
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Gọi `up[k][u]` là tổ tiên thứ 2^k của đỉnh u :

$$\text{up}[0][u] = \text{parent}[u], \quad \text{up}[k][u] = \text{up}[k-1][\text{up}[k-1][u]]$$

- Để tìm tổ tiên thứ K : Phân tích K thành dạng nhị phân. Nếu bit thứ i bật, ta nhảy $u = \text{up}[i][u]$.
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian: Tiền xử lý $O(N \log N)$, mỗi truy vấn $O(\log K)$.

- Cạm bẫy: Khởi tạo giá trị ngoài cây bằng 0 hoặc -1 .

Bài 5: Codeforces 475D — CGCDSSQ

- **Đề bài:** Cho mảng A gồm N số ($N \leq 10^5$, $A_i \leq 10^9$). Cho Q truy vấn ($Q \leq 3 \times 10^5$), mỗi truy vấn gồm một số x . Đếm số lượng cặp (L, R) sao cho $\gcd(A[L \dots R]) = x$.
- **Gợi ý 1 (Nhận xét bản chất):**
 - Với một chỉ số L cố định, khi R tăng dần từ L đến $N - 1$, giá trị $\gcd(A[L \dots R])$ là dãy giảm dần.
 - Hơn nữa, mỗi lần giá trị $\gcd$ giảm, nó phải chia hết cho ít nhất một thừa số nguyên tố ≥ 2 . Do đó, giá trị $\gcd$ chỉ có thể thay đổi tối đa $\log_2(\max A) \approx 30$ lần!
- **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Dùng Sparse Table tính $\gcd$ của mảng A trong $O(N \log N)$.
 - Với mỗi L : Sử dụng tìm kiếm nhị phân để tìm các khoảng $[R_{start}, R_{end}]$ mà $\gcd$ giữ nguyên giá trị g .
 - Cộng số lượng đoạn con $(R_{end} - R_{start} + 1)$ vào bảng băm `std::map<int, long long> ans`.
- **Gợi ý 3 (Độ phức tạp & Cạm bẫy):**
 - Thời gian: $O(N \log N \log(\max A) + Q)$, Bộ nhớ: $O(N \log N)$.
 - Cạm bẫy: Dùng `unordered_map` kèm custom hash để tránh TLE do nhiều giá trị khác nhau.

Bài 6: Codeforces 689D — Friends and Subsequences

- **Đề bài:** Cho hai mảng số nguyên A và B độ dài N ($N \leq 2 \times 10^5$). Đếm số cặp (L, R) ($0 \leq L \leq R < N$) thỏa mãn:

$$\max_{i=L}^R A[i] = \min_{i=L}^R B[i]$$

- **Gợi ý 1 (Nhận xét bản chất):**
 - Đặt $f(R) = \max(A[L \dots R]) - \min(B[L \dots R])$.
 - Khi L cố định và R tăng: $\max(A)$ là hàm không giảm, $\min(B)$ là hàm không tăng $\implies f(R)$ là hàm đơn điệu tăng (non-decreasing).
- **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Xây dựng 2 Sparse Table: `stA` (Max) cho mảng A , và `stB` (Min) cho mảng B .
 - Với mỗi L , vì $f(R)$ đơn điệu, các vị trí R thỏa mãn $f(R) = 0$ sẽ tạo thành một khoảng liên tục $[R_1, R_2]$.
 - Tìm R_1 bằng tìm kiếm nhị phân phần tử đầu tiên có $f(R) \geq 0$.
 - Tìm R_2 bằng tìm kiếm nhị phân phần tử cuối cùng có $f(R) \leq 0$.
 - Nếu $f(R_1) = 0$, số lượng đoạn hợp lệ kết thúc là $R_2 - R_1 + 1$.
- **Gợi ý 3 (Độ phức tạp & Cạm bẫy):**
 - Thời gian: $O(N \log N)$, Bộ nhớ: $O(N \log N)$.
 - Cạm bẫy: Cần thận trường hợp không tồn tại R nào thỏa mãn $f(R) = 0$.

Bài 7: Codeforces 514D — R2D2 and Droid Army

- **Đề bài:** Có N droid ($N \leq 10^5$), mỗi droid có M loại giáp ($M \leq 5$). Bạn có tối đa K viên đạn ($K \leq 10^9$). Để tiêu diệt một đoạn liên tiếp các droid từ L đến R , số đạn cần dùng cho loại giáp thứ j là $\max_{i=L}^R A[i][j]$. Tổng số đạn dùng cho M loại giáp không được vượt quá K . Tìm đoạn droid dài nhất có thể tiêu diệt.
 - **Gợi ý 1 (Nhận xét bản chất):**
 - Chi phí bắn hạ đoạn $[L, R]$ là $\sum_{j=0}^{M-1} \max_{i=L}^R A[i][j]$.
 - Chi phí này đơn điệu tăng theo chiều dài đoạn.
 - **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Dùng M Sparse Table (mỗi thuộc tính giáp một bảng) để truy vấn $\max$ mỗi loại trong $O(1)$.
 - Sử dụng kỹ thuật Hai con trỏ: Duyệt R từ 0 đến $N - 1$, tăng L khi tổng chi phí $> K$.
 - **Gợi ý 3 (Độ phức tạp & Cận bậc):**
 - Thời gian: $O(M \cdot N \log N)$, Bộ nhớ: $O(M \cdot N \log N)$.
 - Cận bậc: In ra chi tiết số lượng đạn từng loại của phương án tối ưu, không chỉ in độ dài.
-

Bài 8: SPOJ THRBL — Catapult that ball

- **Đề bài:** Cho mảng độ cao H của N ngọn đồi ($N, M \leq 50000$). Có M truy vấn: Bắn một viên đạn từ đồi A đến đồi B ($A < B$). Đạn bay tới được B nếu mọi ngọn đồi nằm giữa A và B (tức các đồi $A + 1, \dots, B - 1$) đều có độ cao không vượt quá $H[A]$. Đếm số lượt bắn thành công.
 - **Gợi ý 1 (Nhận xét bản chất):**
 - Điều kiện thành công: $\max_{i=A+1}^{B-1} H[i] \leq H[A]$.
 - Nếu $B = A + 1$, không có ngọn đồi nào ở giữa $\implies$ luôn thành công.
 - **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Xây dựng Sparse Table tìm giá trị lớn nhất trên mảng H .
 - Với mỗi truy vấn, kiểm tra trong $O(1)$ xem $\text{st.query}(A + 1, B - 1) \leq H[A]$.
 - **Gợi ý 3 (Độ phức tạp & Cận bậc):**
 - Thời gian: $O(N \log N + M)$, Bộ nhớ: $O(N \log N)$.
 - Cận bậc: Chú ý trường hợp $A > B$ (bắn ngược hướng) hoặc $A = B$.
-

Bài 9: CSES 1688 — Company Queries II

- **Đề bài:** Cho cây có gốc gồm N đỉnh và Q truy vấn ($N, Q \leq 2 \times 10^5$). Tìm tổ tiên chung gần nhất (LCA) của hai đỉnh u và v .
- **Gợi ý 1 (Nhận xét bản chất):**
- Euler Tour chuyển cây thành dãy các đỉnh được ghé thăm (kích thước $2N - 1$).
- Tổ tiên chung gần nhất của u và v chính là đỉnh có độ sâu nhỏ nhất xuất hiện trên dãy Euler Tour giữa lần đầu tiên xuất hiện của u và lần đầu tiên xuất hiện của v .
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Thực hiện một lượt DFS ghi nhận dãy Euler Tour và mảng độ sâu `depth`.
- Dùng Sparse Table lưu chỉ số RMQ trên mảng `depth` của dãy Euler Tour.
- Mỗi truy vấn $\text{LCA}(u, v)$ được trả lời trong đúng **1 phép truy vấn RMQ** $O(1)$!

- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian: $O(N \log N)$ tiền xử lý, $O(1)$ mỗi truy vấn.
- Cận bậc: Mảng Euler Tour có kích thước lên tới $2N - 1$, khai báo Sparse Table đủ lớn.

Bài 10: AtCoder ABC 254F — Rectangle GCD

- **Đề bài:** Cho hai mảng A và B độ dài N ($N, Q \leq 2 \times 10^5$). Ma trận C kích thước $N \times N$ có $C_{i,j} = A_i + B_j$. Trả lời Q truy vấn tìm gcd của tất cả các phần tử trong hình chữ nhật $[h_1, h_2] \times [w_1, w_2]$.
- **Gợi ý 1 (Nhận xét bản chất):**
- Áp dụng tính chất đại số của gcd: $\gcd(x, y) = \gcd(x, y - x)$.
- Với một hàng i và các cột từ w_1 đến w_2 :

$$\gcd_{j=w_1}^{w_2}(A_i + B_j) = \gcd\left(A_i + B_{w_1}, \gcd_{j=w_1+1}^{w_2}(B_j - B_{j-1})\right)$$

- Mở rộng ra toàn bộ hình chữ nhật:

$$\text{GCD} = \gcd\left(A_{h_1} + B_{w_1}, \gcd_{i=h_1+1}^{h_2}|A_i - A_{i-1}|, \gcd_{j=w_1+1}^{w_2}|B_j - B_{j-1}|\right)$$

- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Không cần dùng Sparse Table 2D!
- Chỉ cần dựng 2 Sparse Table 1D trên mảng hiệu ΔA và ΔB .
- Mỗi truy vấn tính trong $O(1)$ bằng 3 giá trị gcd.
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian: $O(N \log N + Q)$, Bộ nhớ: $O(N \log N)$.
- Cận bậc: Khi $h_1 = h_2$ hoặc $w_1 = w_2$, không có phần tử sai phân nào.

Bài 11: Codeforces 1454F — Array Partition

- **Đề bài:** Cho mảng A gồm N số ($N \leq 2 \times 10^5$). Chia mảng thành 3 đoạn liên tiếp không rỗng $A[1 \dots x]$, $A[x + 1 \dots x + y]$, $A[x + y + 1 \dots N]$ sao cho:

$$\max(A[1 \dots x]) = \min(A[x + 1 \dots x + y]) = \max(A[x + y + 1 \dots N])$$

- **Gợi ý 1 (Nhận xét bản chất):**
- Dựng Sparse Table Min cho đoạn ở giữa. Tiền xử lý tiền tố Max và hậu tố Max.
- Với mỗi vị trí kết thúc x của đoạn 1: Ta biết giá trị cần tìm là $V = \text{pref_max}[x]$.
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Đoạn 3 phải có $\max = V$. Vì hậu tố Max đơn điệu, các vị trí bắt đầu z của đoạn 3 thỏa mãn $\text{suff_max}[z] = V$ tạo thành một khoảng $[z_1, z_2]$.
- Tìm kiếm nhị phân vị trí đoạn giữa $[x + 1, z - 1]$ có $\min = V$ bằng Sparse Table.
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian: $O(N \log N)$, Bộ nhớ: $O(N \log N)$.
- Cận bậc: Cả 3 đoạn đều phải có độ dài ≥ 1 .

Bài 12: Codeforces 1709E — XOR Tree

- **Đề bài:** Cho cây có trọng số đỉnh A_u . Ta muốn thay đổi ít nhất số đỉnh sao cho không có đường đi đơn nào có tổng XOR trọng số các đỉnh bằng 0.
 - **Gợi ý 1 (Nhận xét bản chất):**
 - Đặt X_u là XOR từ gốc tới đỉnh u . Tổng XOR trên đường đi giữa u và v bằng $X_u \oplus X_v \oplus A_{LCA(u,v)}$.
 - Đường đi có tổng XOR bằng 0 khi $X_u \oplus X_v = A_{LCA(u,v)}$.
 - **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Dùng Sparse Table Euler Tour để tìm LCA $O(1)$.
 - Áp dụng kỹ thuật DSU on Tree (Small-to-Large merging) với `std::set` để kiểm tra và chèn XOR trong cây con.
 - **Gợi ý 3 (Độ phức tạp & Chạm bẫy):**
 - Thời gian: $O(N \log^2 N)$.
 - Chạm bẫy: Khi đổi trọng số một đỉnh, ta gán trọng số mới thành một lũy thừa 2 cực lớn chưa từng xuất hiện để triệt tiêu mọi xung đột qua đỉnh đó.
-

Bài 13: SPOJ RPAR — Reduce the Array

- **Đề bài:** Cho mảng tĩnh và các truy vấn tổng hợp trên đoạn với phép toán kết hợp không có phần tử nghịch đảo hoặc nhân đa thức / ma trận modulo M .
 - **Gợi ý 1 (Nhận xét bản chất):**
 - Phép toán không có tính lũy đẳng $\implies$ Sparse Table truyền thống không dùng được $O(1)$.
 - Mảng Tiền Tố không dùng được vì phép toán không có nghịch đảo.
 - **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Cài đặt **Disjoint Sparse Table (DST)**.
 - Phân chia khối theo cây nhị phân, tính tiền tố và hậu tố từ tâm mid .
 - Mọi truy vấn $[L, R]$ trả về trong $O(1)$ qua phép toán `table[MSB(L ^ R)][L] * table[MSB(L ^ R)][R]`.
 - **Gợi ý 3 (Độ phức tạp & Chạm bẫy):**
 - Thời gian: Xây dựng $O(N \log N)$, truy vấn $O(1)$.
-

Bài 14: Codeforces 1304E — 1-Trees and Queries

- **Đề bài:** Cho cây N đỉnh ($N, Q \leq 10^5$). Mỗi truy vấn thêm tạm thời 1 cạnh (x, y) vào cây và hỏi xem có thể đi từ a đến b với đúng k bước hay không (được phép đi lặp lại đỉnh/cạnh).
- **Gợi ý 1 (Nhận xét bản chất):**
 - Thêm một cạnh tạo ra một chu trình.
 - Độ dài đường đi giữa hai đỉnh u, v trên cây gốc là $\text{dist}(u, v) = \text{depth}[u] + \text{depth}[v] - 2 \cdot \text{depth}[\text{LCA}(u, v)]$.
- **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Có 3 đường đi khả dĩ từ a đến b :
 1. Đường đi trực tiếp: $\text{dist}(a, b)$.
 2. Đi qua cạnh mới theo hướng $x \rightarrow y$: $\text{dist}(a, x) + 1 + \text{dist}(y, b)$.

3. Đi qua cạnh mới theo hướng $y \rightarrow x: \text{dist}(a, y) + 1 + \text{dist}(x, b)$.

- Với mỗi đường đi độ dài d , nếu $d \leq k$ và $(k - d)$ chẵn $\implies$ có thể đi được (bằng cách đi đi lại lại trên 1 cạnh).
 - Sử dụng Sparse Table LCA $O(1)$ để tính dist trong $O(1)$.
 - **Gợi ý 3 (Độ phức tạp & Cận bậc):**
 - Thời gian: $O(N \log N + Q)$, Bộ nhớ: $O(N \log N)$.
-

Bài 15: Codeforces 1779F — Even Subarrays

- **Đề bài:** Cho mảng A gồm N số ($N \leq 2 \times 10^5$). Đếm số đoạn con $[L, R]$ sao cho số lượng ước số của tích các phần tử hoặc XOR các phần tử là số lẻ.
 - **Gợi ý 1 (Nhận xét bản chất):**
 - Một số nguyên dương có số lượng ước số lẻ khi và chỉ khi nó là **số chính phương**!
 - XOR prefix sum: $X_R \oplus X_{L-1} = k^2$.
 - **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Giá trị XOR tối đa $< 2^{18} \approx 2.6 \times 10^5$.
 - Duyệt qua mọi số chính phương $v = k^2$ ($0 \leq k \leq \sqrt{2^{18}} \approx 512$).
 - Số đoạn con có XOR là số chính phương bằng $\sum_k \text{count}(X \oplus k^2)$.
 - Trừ số này khỏi tổng số đoạn con $\frac{N(N+1)}{2}$.
 - **Gợi ý 3 (Độ phức tạp & Cận bậc):**
 - Thời gian: $O(N\sqrt{\max A})$.
-

Bài 16: Codeforces 1117G — Recursive Queries

- **Đề bài:** Cho một hoán vị P của $1 \dots N$. Cho Q truy vấn $[L, R]$, hàm $f(l, r)$ tìm vị trí m có P_m lớn nhất trong $[l, r]$, cộng $(r - l + 1)$ vào kết quả và gọi đệ quy $f(l, m - 1)$ và $f(m + 1, r)$. Tính kết quả mỗi truy vấn với $N, Q \leq 10^6$.
 - **Gợi ý 1 (Nhận xét bản chất):**
 - Quá trình đệ quy này chính là cấu trúc của **Cây Descartes (Cartesian Tree)** trên mảng hoán vị!
 - Trong Cartesian Tree, $f(l, r)$ chính là tổng độ sâu của các nút trong cây con sinh bởi đoạn $[l, r]$.
 - Mỗi phần tử i đóng góp vào độ dài đoạn con chừng nào nó còn là cực đại, tức trong khoảng (l_i, r_i) với l_i là vị trí bên trái đầu tiên lớn hơn P_i , và r_i là vị trí bên phải đầu tiên lớn hơn P_i .
 - **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Dùng Monotonic Stack hoặc Sparse Table Index dựng cận l_i, r_i trong $O(N)$.
 - Chuyển bài toán thành tính $\sum_{i=L}^R (\min(R, r_i - 1) - \max(L, l_i + 1) + 1)$.
 - Tách thành 2 truy vấn Offline quét từ trái qua phải và từ phải qua trái sử dụng Fenwick Tree Range Add / Range Sum.
 - **Gợi ý 3 (Độ phức tạp & Cận bậc):**
 - Thời gian: $O(N \log N + Q \log N)$, Bộ nhớ: $O(N + Q)$.
 - Cận bậc: Cần Fast I/O và tối ưu bộ nhớ vì $N, Q = 10^6$.
-



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.